

计算机学院实验报告

实验题目：高级搜索-遗传算法		学号： 202200130048
日期： 12. 17	班级： 22. 6	姓名： 陈静雯
实验目的： 掌握遗传算法的基本原理和实现方法，通过遗传算法求解复杂非线性函数（Rosenbrock 函数）的极大值问题。		
实验环境介绍： 编程语言：Python 开发环境：Anaconda3-5 以上版本		
解决问题的主要思路： 1. 问题分析：Rosenbrock 函数（香蕉函数）是一个经典的非凸优化测试函数，在(1, 1)处取得最小值 0。为求极大值，需要将最小化问题转化为最大化问题。 2. 算法选择：采用遗传算法（Genetic Algorithm, GA）这一启发式优化算法，模拟自然选择和遗传机制。 3. 关键设计： （1）编码方式：使用二进制编码，每个染色体由 32 位二进制串组成（x 和 y 各 16 位） （2）适应度函数：将原函数转换为适应度函数：$fitness = 1/(1 + f(x, y))$ （3）遗传操作：包括选择（轮盘赌）、交叉（单点交叉）、变异（按位变异） 4. 参数设置： 种群大小：100 染色体长度：32 位 迭代次数：100 交叉率：0.8 变异率：0.01		

实验步骤:

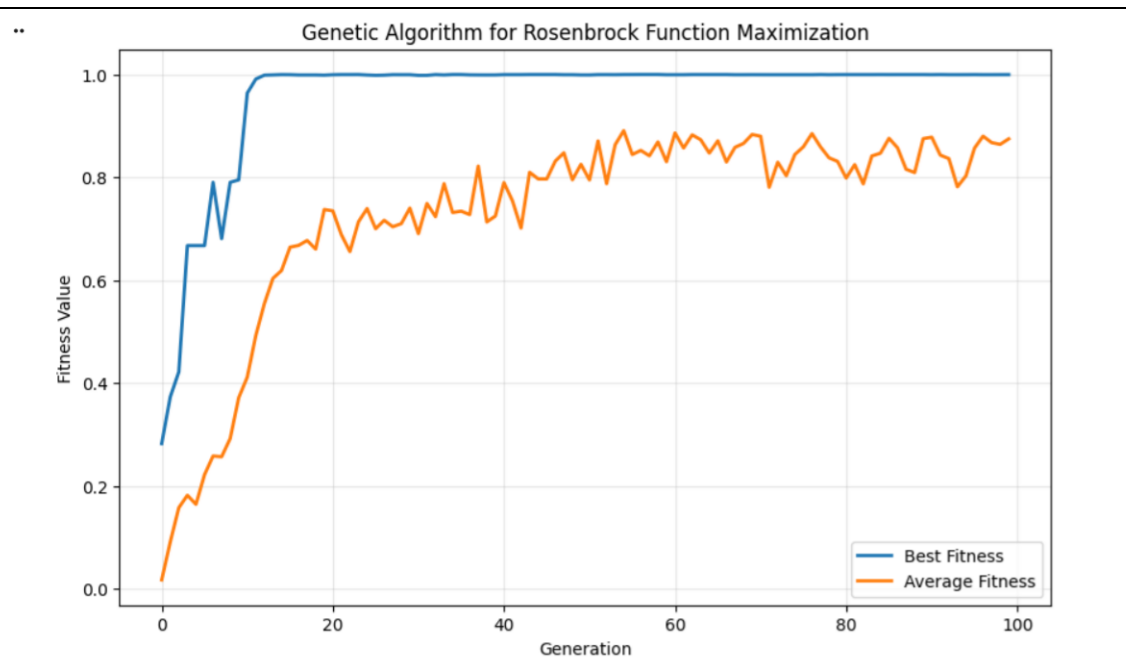
1. 初始化: 随机生成初始种群, 每个个体为 32 位二进制串
2. 适应度评估: 解码染色体, 计算每个个体的适应度值
3. 选择操作: 使用轮盘赌选择法选择父代个体
4. 交叉操作: 对选中的父代进行单点交叉, 生成子代
5. 变异操作: 以一定概率对子代进行变异
6. 更新种群: 用新生成的子代替换旧种群
7. 迭代优化: 重复步骤 2-6, 直到达到最大迭代次数
8. 结果输出: 输出最优解、函数值和适应度值, 绘制收敛曲线

实验结果展示及分析:

```
Generation 0: Best Fitness = 0.2825, Best Solution: x=-0.8022, y=0.8069
Generation 10: Best Fitness = 0.9638, Best Solution: x=0.9816, y=0.9729
Generation 20: Best Fitness = 0.9994, Best Solution: x=0.9759, y=1.0220
Generation 30: Best Fitness = 0.9982, Best Solution: x=0.4536, y=0.8941
Generation 40: Best Fitness = 0.9996, Best Solution: x=0.9716, y=0.9800
Generation 50: Best Fitness = 0.9990, Best Solution: x=0.9816, y=0.9525
Generation 60: Best Fitness = 0.9995, Best Solution: x=0.9814, y=0.9425
Generation 70: Best Fitness = 0.9995, Best Solution: x=0.9810, y=0.9505
Generation 80: Best Fitness = 0.9997, Best Solution: x=0.9773, y=0.9738
Generation 90: Best Fitness = 0.9996, Best Solution: x=0.9771, y=0.9903
```

Final Results:

```
Optimal solution: x = 0.986609, y = 0.974296
Function value: f(0.986609, 0.974296) = 0.000260
Fitness value: 0.999740
```



结果分析：

1. 收敛性分析：

- 初始代适应度仅为 0.2825，经过 10 代迭代迅速提升至 0.9638
- 20 代后适应度达到 0.9994，接近最优值
- 后续迭代在 0.999 附近波动，表明算法已基本收敛

2. 解的质量：

- 最优解为 $x=0.986609$ ， $y=0.974296$ ，非常接近理论最优解 (1, 1)
- 函数值 $f(x, y)=0.000260$ ，接近最小值 0
- 适应度值 0.999740，接近最大值 1

3. 算法性能：

- 算法在 20 代左右已找到较优解，体现了遗传算法快速收敛的特点
- 后期迭代中解的质量有小幅波动，这是遗传算法随机性的正常表现
- 最终结果验证了遗传算法求解复杂优化问题的有效性

关键代码说明：

1. 适应度函数设计

```
def fitness_function(x, y):  
    return 1 / (1 + rosenbrock(x, y))
```

说明：

- **功能：**将 Rosenbrock 函数的最小化问题转化为最大化问题
- **原理：**
 - Rosenbrock 函数在 (1, 1) 处取得最小值 0
 - 通过取倒数，将函数值映射到 (0, 1] 区间
 - 函数值越小 → 适应度越大，符合遗传算法求极大值的需求
- **优点：**避免负值，数值稳定

2. 染色体编码与解码

```
def decode_chromosome(chromosome):  
    x_bits = chromosome[:16] # 前 16 位表示 x  
    y_bits = chromosome[16:] # 后 16 位表示 y  
    x_int = int(x_bits, 2)    # 二进制转十进制  
    y_int = int(y_bits, 2)  
    # 映射到[-2.048, 2.048]区间  
    x = -2.048 + (x_int / 65535) * 4.096  
    y = -2.048 + (y_int / 65535) * 4.096  
    return x, y
```

说明:

- **编码策略:** 二进制编码, 每个变量用 16 位表示
- **映射公式:** $value = min + (int_value / (2^{16} - 1)) * (max - min)$
 - 16 位二进制最大值为 65535 ($2^{16}-1$)
 - 将 [0, 65535] 线性映射到 [-2.048, 2.048]
- **优点:** 编码简单, 解码方便, 搜索空间覆盖完整定义域

3. 选择操作 (轮盘赌选择)

```
def selection(population, fitness_values):  
    total_fitness = sum(fitness_values)  
    probabilities = [f/total_fitness for f in fitness_values]  
    selected_indices = np.random.choice(  
        range(len(population)),  
        size=len(population),  
        p=probabilities,  
        replace=True  
    )  
    return [population[i] for i in selected_indices]
```

说明:

- **算法原理:** 适应度比例选择法
- **概率计算:** $P(i) = fitness(i) / \sum fitness$
- **实现方式:** 使用 `numpy.random.choice` 函数
- **参数:**
 - `p`: 概率分布, 适应度越高被选中概率越大
 - `replace=True`: 允许重复选择 (适应度高的个体可能被多次选中)
- **作用:** 模拟自然选择, 保留优秀个体

4. 交叉操作 (单点交叉)

```
def crossover(parent1, parent2, crossover_rate):  
    if np.random.rand() < crossover_rate:  
        crossover_point = np.random.randint(1, len(parent1)-1)  
        child1 = parent1[:crossover_point] + parent2[crossover_point:]  
        child2 = parent2[:crossover_point] + parent1[crossover_point:]  
    return child1, child2
```

```
else:
    return parent1, parent2
```

说明：

- **交叉方式：**单点交叉
- **执行条件：**以 crossover_rate（默认 0.8）的概率执行
- **过程：**
 1. 随机选择交叉点（1 到染色体长度-1）
 2. 交换两个父代交叉点后的基因片段
 3. 生成两个子代
- **作用：**组合不同个体的优秀基因，产生新解

5. 变异操作（按位变异）

```
def mutation(chromosome, mutation_rate):
    chromosome = list(chromosome)
    for i in range(len(chromosome)):
        if np.random.rand() < mutation_rate:
            chromosome[i] = '1' if chromosome[i] == '0' else '0'
    return ''.join(chromosome)
```

说明：

- **变异方式：**按位变异（位翻转）
- **执行条件：**每个基因位以 mutation_rate（默认 0.01）的概率变异
- **变异操作：**二进制位 0↔1 翻转
- **作用：**
 - 引入新的基因，增加种群多样性
 - 避免算法过早收敛到局部最优
 - 帮助跳出搜索空间的局部最优区域

6. 主循环逻辑

```
for generation in range(generations):
    # 1. 评估适应度
    fitness_values = []
    for chrom in population:
        x, y = decode_chromosome(chrom)
        fitness_values.append(fitness_function(x, y))

    # 2. 记录历史数据
    best_fitness = max(fitness_values)
    avg_fitness = np.mean(fitness_values)
    best_fitness_history.append(best_fitness)
    avg_fitness_history.append(avg_fitness)

    # 3. 遗传操作
    selected_population = selection(population, fitness_values) # 选
```

择

```
# ... 交叉和变异操作
```

```
# 4. 更新种群
```

```
population = new_population
```

说明：

- **算法流程：**评估→选择→交叉→变异→更新
- **迭代控制：**固定迭代次数（100 代）
- **监控指标：**
 - `best_fitness_history`：记录每代最佳适应度
 - `avg_fitness_history`：记录每代平均适应度
- **收敛判断：**通过观察适应度曲线判断收敛情况

7. 参数设置与说明

```
def genetic_algorithm(pop_size=100, chrom_length=32, generations=100,  
                      crossover_rate=0.8, mutation_rate=0.01):
```

参数说明：

1. `pop_size=100`：种群大小
 - 影响搜索能力，过大计算开销大，过小易早熟
2. `chrom_length=32`：染色体长度
 - 32 位二进制，x 和 y 各 16 位
 - 分辨率： $4.096/65535 \approx 0.0000625$
3. `generations=100`：迭代次数
 - 控制算法运行时间，需平衡计算成本和解的质量
4. `crossover_rate=0.8`：交叉概率
 - 控制基因重组频率，通常 0.6-0.9
5. `mutation_rate=0.01`：变异概率
 - 控制种群多样性，通常 0.001-0.05

8. 结果可视化代码

```
python  
plt.figure(figsize=(10, 6))  
plt.plot(best_history, label='Best Fitness', linewidth=2)  
plt.plot(avg_history, label='Average Fitness', linewidth=2)  
plt.xlabel('Generation')  
plt.ylabel('Fitness Value')  
plt.title('Genetic Algorithm for Rosenbrock Function Maximization')  
plt.legend()  
plt.grid(True, alpha=0.3)  
plt.show()
```

说明：

- **可视化目的：**监控算法收敛过程
- **曲线意义：**
 - 最佳适应度曲线：反映算法找到的最优解质量
 - 平均适应度曲线：反映整个种群的进化水平

- **收敛判断：**
 - 曲线趋于平缓表示收敛
 - 最佳适应度接近 1 表示找到接近最优的解

附件：关键程序代码